

week_12\week12\src\main\java\week12\Board2D.java

```
1  package week12;
2  import java.io.*;
3
4  /**
5   * The Board2D class represents a 2D tic-tac-toe board.
6   * It provides methods for printing the board, checking if the board is full,
7   * checking if a cell is empty,
8   * getting the position of a cell based on its number, placing a marker on the
9   * board, and checking for a winner.
10  * The class uses a 2D array to represent the board and implements the necessary
11  * logic to determine the game state.
12  */
13
14 public class Board2D extends Board {
15     private char board[][];
16     private PrintStream out;
17
18     /**
19      * Constructor for Board2D class.
20      * Initializes the board as a 2D array of characters and fills it with '0' to
21      * represent empty cells.
22      * @param out
23      */
24     public Board2D(PrintStream out){
25         this.out = out;
26         board = new char[Constants.ROW][Constants.COL];
27         for(int i = 0; i < Constants.ROW; i++){
28             for(int j = 0; j < Constants.COL; j++){
29                 board[i][j] = '0';
30             }
31         }
32     }
33
34     /**
35      * Prints the current state of the board.
36      */
37     @Override
38     public void print(){
39         for(int i = 0; i < Constants.ROW; i++){
40             for(int j = 0; j < Constants.COL; j++){
41                 out.printf("| %s ", "" + board[i][j]);
42             }
43             out.println("|");
44         }
45     }
46
47     /**
48      * Returns the total number of cells on the board.
49      * @return the total number of cells
50      */
51     @Override
52     public int getTotalCells(){
53         return Constants.ROW * Constants.COL;
54     }
55 }
```

```
49     }
50
51     /**
52     * Checks if the board is full.
53     * @return true if the board is full, false otherwise
54     */
55     @Override
56     public boolean isBoardFull(){
57         for(int i = 0; i < Constants.ROW; i++){
58             for(int j = 0; j < Constants.COL; j++){
59                 if(board[i][j] == '0'){
60                     return false;
61                 }
62             }
63         }
64         return true;
65     }
66
67     /**
68     * Checks if a cell is empty.
69     * @param pos the position of the cell to check
70     * @return true if the cell is empty, false otherwise
71     */
72     @Override
73     public boolean isEmpty(Position pos){
74         return board[pos.getRow()][pos.getCol()] == '0';
75     }
76
77     /**
78     * Gets the position of a cell based on its box number.
79     * The box number is a 1-based index that corresponds to the cell's position
80     on the board.
81     * @param boxNumber the number of the cell (1-based index)
82     * @return the Position object representing the cell's position on the board
83     */
84     @Override
85     public Position getCellPosition(int boxNumber){
86         int row = (boxNumber - 1) / Constants.COL;
87         int col = (boxNumber - 1) % Constants.COL;
88         return new Position(row, col);
89     }
90
91     /**
92     * Places a marker on the board at the specified position.
93     * @param pos the position where the marker should be placed
94     * @param marker the marker to place on the board
95     */
96     @Override
97     public void placeMarker(Position pos, char marker){
98         board[pos.getRow()][pos.getCol()] = marker;
99     }
100
101     /**
102     * Checks if there is a winner on the board.
```

```

102     * @return the marker of the winning player, or '0' if there is no winner
103     */
104     @Override
105     public char checkWinner(){
106         // Left → Right
107         for(int i = 0; i < Constants.ROW; i++){
108             for(int j = 0; j ≤ Constants.COL - Constants.WIN_LENGTH; j++){
109                 if(checkLine(new Position(i, j), 0, 1)){
110                     return board[i][j];
111                 }
112             }
113         }
114         // Top → Bottom
115         for(int j = 0; j < Constants.COL; j++){
116             for(int i = 0; i ≤ Constants.ROW - Constants.WIN_LENGTH; i++){
117                 if(checkLine(new Position(i, j), 1, 0)){
118                     return board[i][j];
119                 }
120             }
121         }
122
123         // Diagonal
124         for(int i = 0; i ≤ Constants.ROW - Constants.WIN_LENGTH; i++){
125             for(int j = 0; j ≤ Constants.COL - Constants.WIN_LENGTH; j++){
126                 if(checkLine(new Position(i, j), 1, 1)){
127                     return board[i][j];
128                 }
129             }
130         }
131
132         // Anti-diagonal
133         for(int i = 0; i ≤ Constants.ROW - Constants.WIN_LENGTH; i++){
134             for(int j = Constants.WIN_LENGTH - 1; j < Constants.COL; j++){
135                 if(checkLine(new Position(i, j), 1, -1)){
136                     return board[i][j];
137                 }
138             }
139         }
140         return '0';
141     }
142
143     /**
144     * Checks if a line has the same marker.
145     * @param startPos the starting position of the line
146     * @param steps the steps to move in each direction
147     * @return true if the line has the same marker, false otherwise
148     */
149     @Override
150     protected boolean checkLine(Position startPos, int... steps){
151         int startRow = startPos.getRow();
152         int startCol = startPos.getCol();
153         int rowStep = steps[0];
154         int colStep = steps[1];
155         char marker = board[startRow][startCol];

```

```

156         if(marker == '0'){
157             return false;
158         }
159
160         for(int k = 1; k < Constants.WIN_LENGTH; k++){
161             int row = startRow + k * rowStep;
162             int col = startCol + k * colStep;
163
164             // Out of Bound
165             if(row < 0 || row ≥ Constants.ROW || col < 0 || col ≥ Constants.COL)
166         {
167             return false;
168         }
169
170         // Not the same marker
171         if(board[row][col] ≠ marker){
172             return false;
173         }
174         return true;
175     }
176
177     /**
178     * Take the board and make it into a string for transportation in a network
179     * Example, which I hope will look like this: 000010002
180     * @return a string of characters represent each cell's state in order from
181 box 1 - the final cell
182     * which is 9 btw ;)
183     */
184     public String networkString(){
185         StringBuilder stringBuilder = new StringBuilder();
186
187         for(int i = 1; i ≤ getTotalCells(); i++){
188             Position position = getCellPosition(i);
189             char marker = board[position.getRow()][position.getCol()];
190             stringBuilder.append(marker);
191         }
192         return stringBuilder.toString();
193     }
194
195     /**
196     * The reverse of networkString method, this method take the string from the
197 network
198     * and then rebuild it
199     * @param string a 9-character-ish (can be more but well this is a 3 x 3
200 board)
201     * containing the marker for boxes 1-9
202     */
203     public void loadString(String string){
204         for (int i = 0; i < string.length(); i++){
205             char marker = string.charAt(i);
206             int boxNumber = i + 1;
207             Position position = getCellPosition(boxNumber);

```

```
206 |           placeMarker(position, marker);
207 |       }
208 |   }
209 | }
210 |
```